Software Implementation and Testing Document

For

Group 7

Version 2.0

Authors:
Chiara Savoretti
Teena Singh
Ramon Ortega
Samuel Marcano
Francisco de la Espriella

1. Programming Languages (5 points)

The Chupi application utilizes **Python** and **JavaScript** as its primary programming languages.

- **Python** is used for the backend development of the application, specifically through the Django framework. It handles core application logic, including player management, game flow control, and implementation of the various game modes (e.g., Truth or Drink, Imposter, etc.). Python was chosen due to its simplicity, readability, and strong support for rapid development, as well as its compatibility with object-oriented design principles.
- **JavaScript** is used on the frontend to enhance interactivity within the web application. It manages dynamic user interface behavior such as handling user inputs, updating the UI without full page reloads, and improving overall responsiveness. JavaScript was selected because it is the standard language for client-side web development and integrates seamlessly with HTML and CSS.

Additionally, **HTML** and **CSS** are used for structuring and styling the user interface, ensuring a responsive and user-friendly design consistent with the system's mobile compatibility requirements.

2. Platforms, APIs, Databases, and other technologies used (5 points)

The Chupi project uses a combination of frameworks and web technologies to implement both backend and frontend components:

- **Django (Web Framework):**
  Django serves as the primary backend platform. It is responsible for routing, handling HTTP requests, managing application logic, and providing an administrative interface (Django Admin) for testing and data manipulation. Django was chosen due to its built-in features, scalability, and strong support for rapid web development.
- **Frontend Web Stack (HTML, CSS, JavaScript):**
  These technologies are used to build the user interface of the application. HTML provides structure, CSS ensures responsive design (meeting screen-size requirements from 320px to 1024px), and JavaScript adds interactivity.
- **Python Virtual Environments (venv):**
  Used to isolate dependencies and maintain a consistent development environment.
- No external APIs are currently integrated into the system.

3. Execution-based Functional Testing (10 points)

Execution-based functional testing was performed by actively running the application and verifying that all functional requirements outlined in the Requirements Document (RD) were satisfied.

Testing was conducted through two primary methods:

- **Live Application Testing:**
  The team interacted directly with the web application, simulating real user behavior to verify core functionalities. For example:
  - Ensuring the system correctly accepts between 2 and 10 players (FR-01).
  - Verifying that each player must enter a unique, non-empty name (FR-02).
  - Confirming that users can select a game mode from the main menu and that the selected game launches properly (FR-03, FR-04, FR-05).
  - Testing that turn-based logic cycles correctly between players (FR-06).
- **Django Admin Testing:**
  The Django admin interface was used to directly manipulate and inspect application data, allowing validation of backend functionality such as player creation and session handling.

These methods ensured that all major system features behaved as expected under normal usage conditions and aligned with the defined use cases (e.g., player registration and game selection workflows).

4. Execution-based Non-Functional Testing (10 points)

Execution-based non-functional testing focused on verifying that the system met performance, usability, and design requirements specified in the RD.

Testing included:

- **Usability Testing (NFR-02):**
  A third-party user with no prior experience was asked to interact with the application. The goal was to confirm that the system could be used intuitively without external instructions. Feedback indicated that the interface was clear and easy to navigate.
- **Performance Testing (NFR-01):**
  The application was tested to ensure that user inputs were processed within an acceptable time frame (approximately under 1 second) during normal operation.
- **Responsive Design Testing (NFR-06):**
  Browser developer tools (inspect element) were used to simulate various screen sizes. This ensured that the interface rendered correctly on devices ranging from small mobile screens (320px) to larger tablets (1024px).
- **Error Handling (NFR-07):**
  The system was tested with invalid inputs (e.g., non-numeric entries, out-of-range values) to ensure that it handled errors gracefully and re-prompted the user rather than crashing.

These tests confirmed that the application meets key non-functional requirements related to usability, responsiveness, and robustness.

5. Non-Execution-based Testing (10 points)

Non-execution-based testing was performed through structured analysis and review of the codebase without running the application.

This included:

- **Code Reviews and Inspections:**
  Team members reviewed each other's code to ensure adherence to project standards and requirements. This included verifying:
  - Proper use of object-oriented design (e.g., separation between core logic and game modules, as required by NFR-03).
  - Implementation of a common base class (`GameBase`) and consistent subclass structure (NFR-04).
  - Readability and maintainability of the code.
- **Standards Compliance Checks:**
  The codebase was evaluated against the requirements document to ensure all specified priorities were addressed appropriately.
- **Documentation Review:**
  The team ensured that code includes appropriate structure and organization, supporting future extensibility (NFR-09), and aligns with the system architecture described in the class diagrams.

These methods helped identify potential issues early, improve code quality, and ensure the system design aligns with the specified architecture and requirements.